

S

ShopEZ

Frontend Documentation

Angular 19

.NET 9 (C#)

SQL Server

YARP Gateway

Docker

JWT Auth

xUnit + Jasmine

Project Details

Trainee

Yadavali Jayasree

Batch

B3_Angular

Trainer

Narasimha Rao

Email

yadavalijayasree9@gmail.com

Program

UpGrad CTS Pre-Assessment Hiring

Submission

May 2026

1. Project Overview

ShopEZ is a full-stack e-commerce Single Page Application built as a Capstone Project for the UpGrad CTS Pre-Assessment Hiring Program. The Angular frontend delivers a modern Amazon-inspired shopping experience, communicating with a .NET microservices backend through a YARP API Gateway.

The application covers the complete shopping lifecycle — from browsing and filtering products, managing a cart and wishlist, completing a 3-step checkout, and tracking orders — to a full admin panel for managing products, categories, orders, and users.

Component	Technology / Value
Framework	Angular (Standalone Components)
Language	TypeScript 5
Styling	Bootstrap 5 + Bootstrap Icons + Custom CSS
State Management	RxJS BehaviorSubject
HTTP	HttpClient + AuthInterceptor
Auth	JWT — localStorage persistence
Cart / Wishlist	localStorage (client-side)
API Gateway	http://localhost:7000
Testing	Karma + Jasmine — 58 specs, 0 failures
Build Tool	Angular CLI / Webpack

1.1 Key Features

Feature	Description
Home Page	Hero banner, Shop by Category grid, promo banners, Featured Products section
Product Listing	Responsive grid, client-side filtering by category / price / rating, sorting
Keyword Search	URL query param driven search (?keyword=...), filtered in real-time on client
Product Details	Image, description, quantity selector, Add to Cart / Buy Now, reviews form, related products
Shopping Cart	localStorage-persisted with live subtotal, qty controls, save for later
Checkout Wizard	3-step flow: Delivery Address → Payment Method → Review & Place Order
Order History	Paginated list with status badges, product thumbnails, track / cancel actions
Order Tracking	Visual 4-step timeline: Placed → Confirmed → Shipped → Delivered
Wishlist	localStorage-persisted, toggle heart on any product, Add to Cart shortcut

Feature	Description
Authentication	Register / Login with JWT, route guards (authGuard, adminGuard, guestGuard)
Admin Panel	Dashboard stats, Manage Products (image upload), Categories CRUD, Orders status, Users list
Responsive Design	Bootstrap 5 grid — works on mobile (offcanvas filter), tablet, and desktop

2. Home Page

2.1 Hero Banner + Shop by Category

Dark gradient background with the ShopEZ brand name. Two CTA buttons — 'Shop Now' and 'Browse All' — both route to /products. Categories are loaded from GET /api/categories. Each card shows a Bootstrap icon and a coloured circular background. If the API returns empty, 8 default categories are shown as a fallback.

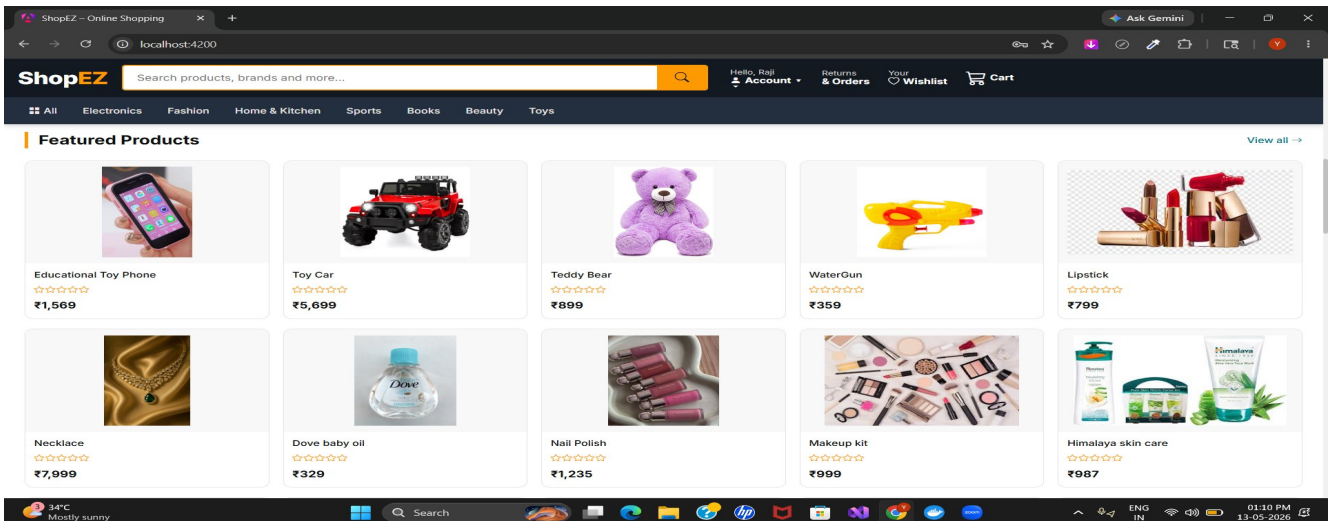


Figure 1 & 2: Home page — Featured Products grid showing real product images (Educational Toy Phone, Toy Car, Teddy Bear, WaterGun, Lipstick). ShopEZ header with Search, Account, Wishlist, Cart.

2.2 Featured Products Grid

Loads the first 10 products from the API using a 5-column responsive grid (row-cols-2 on mobile → row-cols-lg-5 on desktop). A safety timeout of 8 seconds ensures the spinner never freezes the page if the backend is slow.

2.3 Promo Banners

Three coloured promotional cards — Top Electronics, Fashion & Style, Home & Kitchen — each with an icon, tag label, title, and a 'Shop now' link pointing to the filtered products page for that category.

3. Authentication

3.1 Login Page

The Login component uses a template-driven form with ngModel. On submit, AuthService.login() POSTs to /api/auth/login via the API Gateway. The normalizeAuthResponse() method handles both response shapes the backend may return.

On success, handleAuthSuccess() stores the JWT in localStorage under 'shopez_token' and the user object under 'shopez_user', emits the user through the BehaviorSubject, then navigates to returnUrl (or home if no returnUrl).

Detail	Value
Seeded Admin Email	admin@shopez.com
Seeded Admin Password	Admin@123
guestGuard	Prevents already-logged-in users from accessing /login — redirects them to home
Role Normalisation	role is uppercased on store so 'admin', 'Admin', 'ADMIN' all pass isAdmin()

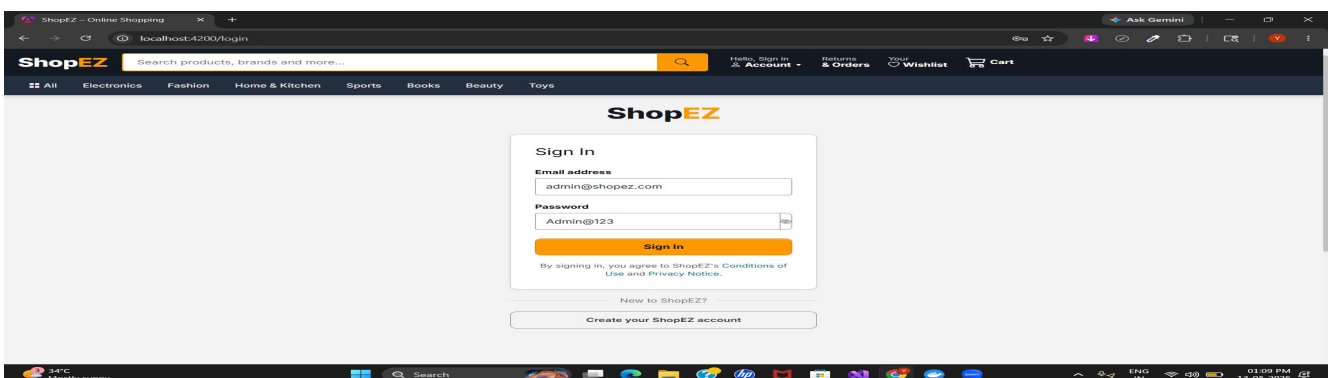


Figure 3: Login page — email (admin@shopez.com) and password fields, Sign In button, Create ShopEZ account link.
Admin credentials: admin@shopez.com / Admin@123

3.2 Register Page

The Register component collects name, mobile number, email, password, and confirm password.

Feature	Details
Password Strength Meter	Scores 0–4 based on: length ≥ 8 , uppercase, lowercase, digit, special character. Labels: Weak / Fair / Good / Strong
Phone Validation	Pattern /^[6-9]\d{9}\$/ — Indian mobile numbers starting with 6, 7, 8, or 9
Password Match	Custom validator compares password and re-enter password fields
isLoading Flag	Disables the submit button and shows 'Registering...' during the API call
Validation Timing	Errors fire after the user touches and leaves each field (touched + dirty)

4. Product Listing

4.1 Products Page with Filters

ProductList loads ALL products once from the backend into allProducts[] on initialisation, then applies all filters in memory. This design makes every filter change instantaneous — no additional HTTP requests.

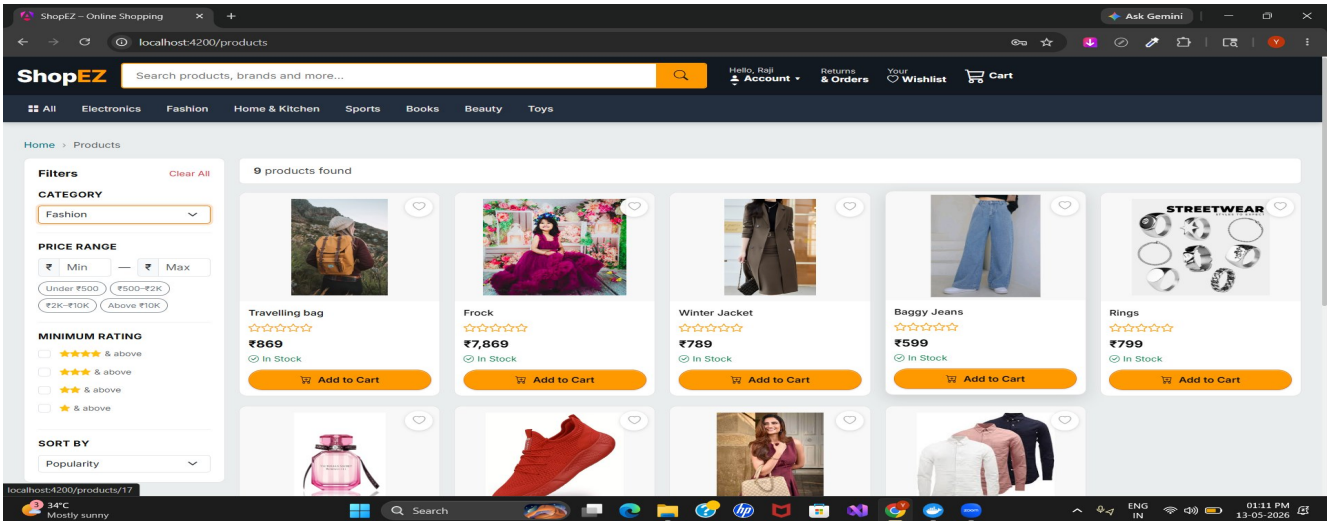


Figure 5: Product listing — Fashion products filtered by category and price range ₹500–₹2K, sorted by Price: High to Low. Filter sidebar shows Category dropdown, Price Range, Minimum Rating, Sort By

Client-Side Filtering Strategy

Step	Action
1. ngOnInit	Fetch all products → store in allProducts[]
2. Filter Change	Any filter change → call applyFilters() on allProducts[]
3. Apply Sequence	category → keyword → minPrice → maxPrice → minRating → sort
4. Render	Result stored in products[] — the rendered grid array. No re-fetch on filter change.

Filter	UI Control	How It Works
Category	Dropdown (select)	Loaded from GET /api/categories — selectedCategoryId compared to product.categoryId
Price Range	Number inputs + preset chips	Under ₹500 ₹500–₹2K ₹2K–₹10K Above ₹10K. Sets minPrice / maxPrice
Minimum Rating	Checkbox list	4★ & above → minRating=4, filters products where rating >= minRating
Sort By	Dropdown	price_asc / price_desc sorts products[]. 'popularity' = default backend order

4.2 Keyword Search

When the user types in the header search bar and presses Enter, the router navigates to `/products?keyword=`. The `ProductList` component reads the keyword from `ActivatedRoute.queryParams` and re-runs `applyFilters()`. The keyword filter checks `product.name` and `product.description` using case-insensitive `.includes()`.

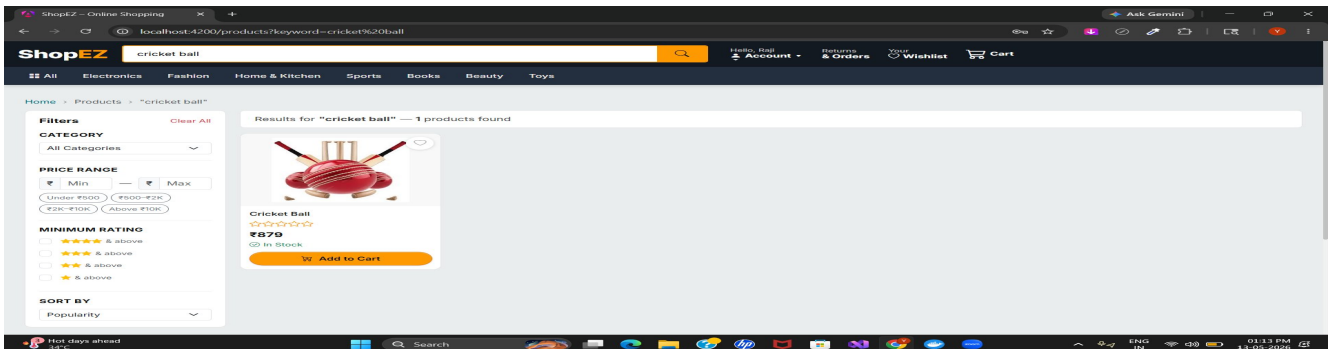


Figure 6: Search results for 'cricket ball' — 1 product found (Cricket Ball ₹879, In Stock). URL: `/products?keyword=cricket%20ball`. Filter sidebar resets to All Categories.

4.3 Image Lazy Loading & Wishlist Heart Button

All product images use the HTML native `loading="lazy"` attribute. The browser only downloads images when they are about to scroll into the viewport. The `onImgError` handler catches broken image URLs and replaces them with a placeholder. Every product card has a wishlist toggle button — clicking calls `wishlistService.toggleWishlist(product)` and the icon switches between bi-heart (outline) and bi-heart-fill (red).

5. Product Details

The ProductDetails component fetches data from GET /api/products/:id. Three data calls happen concurrently on ngOnInit: loadProduct(id), loadReviews(id), and loadRelated(id).

Feature	Description
Quantity Selector	Dropdown showing options 1 to min(stock, 10). Disabled when stock === 0
Buy Now	Adds the product to cart with selected quantity and immediately navigates to /checkout
Wishlist Toggle	Calls wishlistService.toggleWishlist(product). Shows success/info alert
Review Form	Visible only when logged in. Reactive form with rating (click-to-select stars), title, comment
Optimistic Update	On submit, the new review is added to the top of the list immediately before server confirmation
ChangeDetectorRef	detectChanges() called explicitly to force Angular to update the DOM after async operations

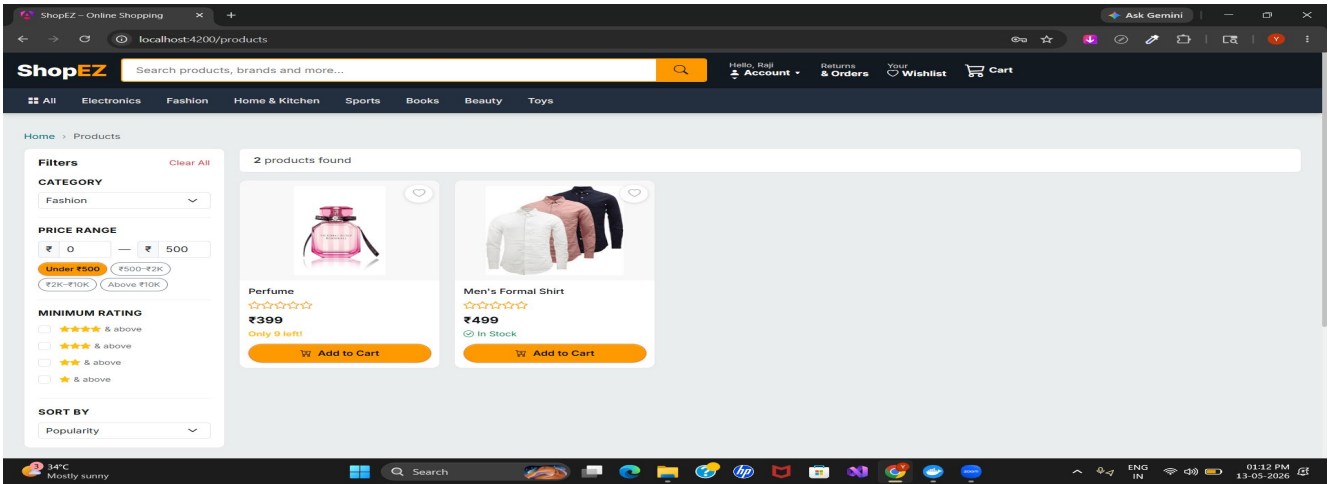


Figure 5b: Product listing — Fashion category filtered Under ₹500. Shows Perfume (₹399, Only 9 left!) and Men's Formal Shirt (₹499, In Stock). Price preset chip 'Under ₹500' highlighted in orange.

6. Shopping Cart

The Cart page subscribes to `cartService.cart$` using the `takeUntil(destroy$)` pattern. The subscription automatically unsubscribes when the component is destroyed, preventing memory leaks.

Each cart item displays: product thumbnail, product name, In Stock badge, quantity control (– disabled at 1, + disabled at `product.stock`), Delete | Save for later links, and item subtotal (`price × quantity`). The summary sidebar shows subtotal, a 'Proceed to Buy' button, and an EMI estimate.

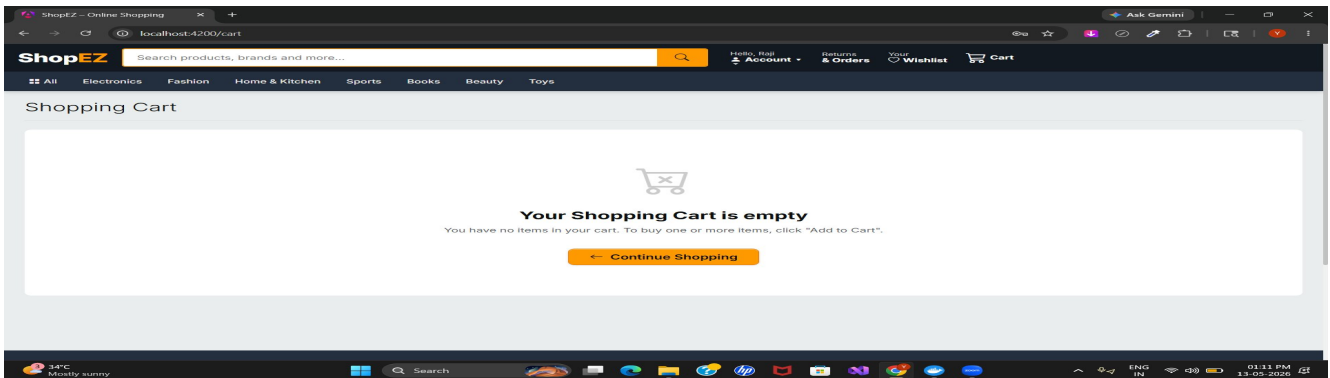


Figure 8: Empty cart state — 'Your Shopping Cart is empty', Continue Shopping button routing to `/products`. Shown when `cart.items.length === 0`.

6.1 CartService — BehaviorSubject Pattern

CartService uses a BehaviorSubject which always holds the current cart state. Any component can subscribe to `cart$` and receive the latest state immediately. Every mutation recalculates totals and persists to `localStorage` atomically. The header's cart badge and the cart page both subscribe to `cart$` — when an item is added anywhere in the app, both update simultaneously without any communication between components.

7. Wishlist

WishlistService uses the same BehaviorSubject pattern as CartService. The wishlist state is stored in localStorage under 'shopez_wishlist'. The full product object is stored in each WishlistItem so product name, price, and category display correctly even if the product list has not been loaded.

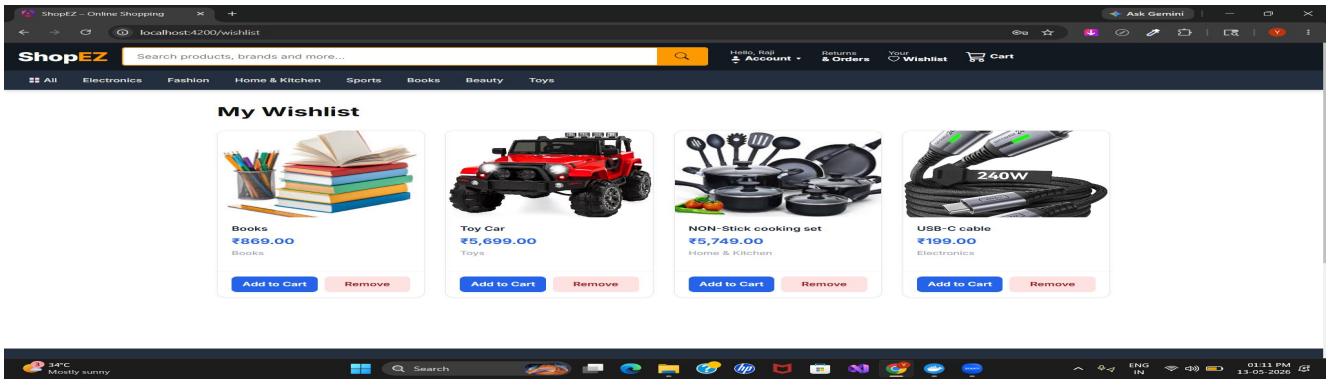


Figure 10: Wishlist page — 4 saved items: Books (₹869), Toy Car (₹5,699), NON-Stick cooking set (₹5,749), USB-C cable (₹199). Add to Cart (blue) and Remove (pink) buttons.

Method	Action
toggleWishlist(product)	Adds to wishlist if not present; removes if already there
isInWishlist(productId)	Returns boolean — used to show filled red heart vs. outline heart icon
remove(productId)	Removes item by productId. Called by the 'Remove' button on the wishlist page
wishlist\$	Observable — the Wishlist page component subscribes to this for live updates

8. Checkout — 3-Step Wizard

The Checkout component guides the user through three steps. The active step number is tracked in `currentStep` (1, 2, or 3). Each step's content is shown using `*ngIf='currentStep === N'` and the completed step shows a summary badge in the header with a 'Change' link.

Step	Name	Details
1	Delivery Address	ReactiveForm with full validation. Supports saved addresses or adding a new address. Dropdown of all 36 Indian states + UTs. Fields: Full Name, Phone (Indian 10-digit regex), Address Line 1 & 2, City, State, PIN Code
2	Payment Method	UPI (with UPI ID validation: <code> /^[w.\-]{3,}@[a-zA-Z]{3,}\$ /</code>) or Cash on Delivery. Validation prevents proceeding with no selection
3	Review & Place Order	Shows all cart items with images, quantities, and totals. 'Place Order' POSTs to <code>/api/orders</code> and redirects to <code>/orders?placed=true</code> on success

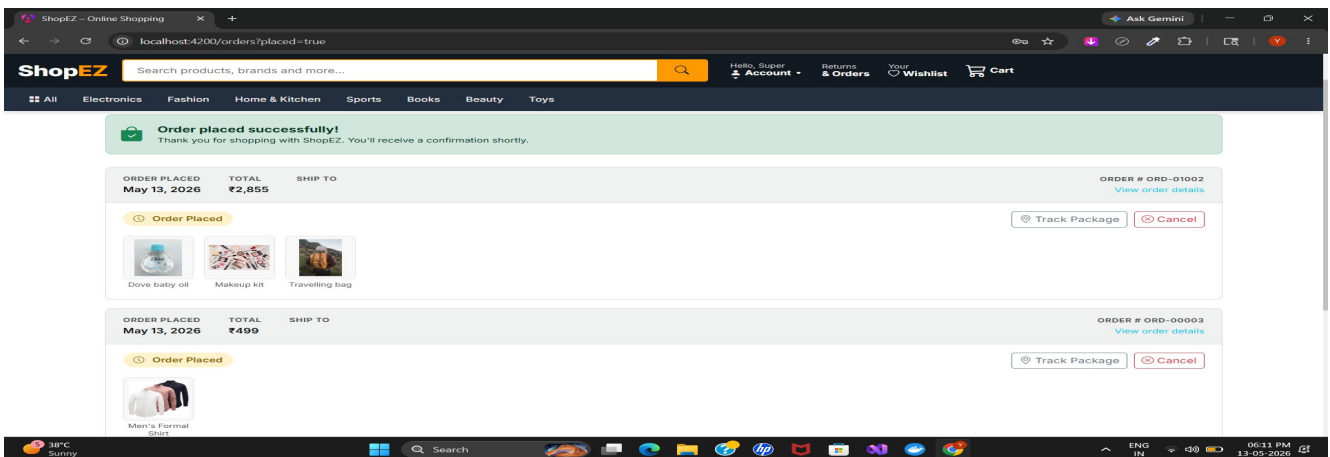


Figure 13: Order History after successful checkout — 'Order placed successfully!' green banner. ORD-01002 (₹2,855 — Dove baby oil, Makeup kit, Travelling bag) and ORD-00003 (₹499 — Men's Formal Shirt). Track Package and Cancel buttons.

8.1 Order Placement Flow

- `cartService.clearCart()` — empties the cart and removes `localStorage` entry
- `router.navigate(['/orders'], { queryParams: { placed: 'true' } })`
- `OrderHistory` detects `placed=true` and shows the green success banner at the top

9. Order History & Order Tracking

9.1 Order History

The order history page loads from GET /api/orders/my-orders. Key features:

- justPlaced flag — reads placed=true from URL query params. Shows the green success banner only once
- Status badges — colour-coded pills: orange (Order Placed), blue (Confirmed), teal (Shipped), green (Delivered), red (Cancelled)
- Product thumbnails — shows up to 4 product images per order. A '+N more' pill appears if there are more than 4 items
- Cancel button — visible only when status === 'PENDING'. Optimistically updates the UI to CANCELLED immediately
- Pagination — pageArray getter generates page buttons. loadPage(n) fetches page n from the API

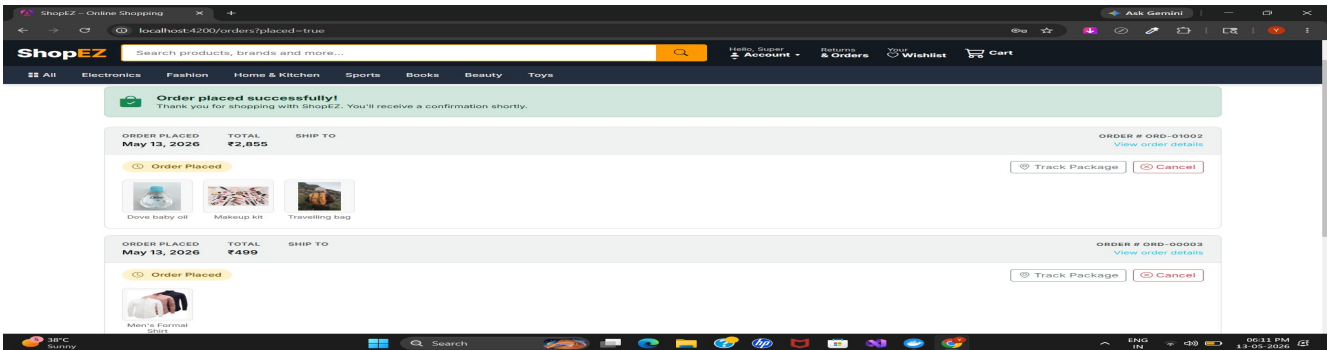


Figure 13: Order History — 'Order placed successfully!' green banner. ORD-01002 (₹2,855, Dove baby oil, Makeup kit, Travelling bag) and ORD-00003 (₹499, Men's Formal Shirt). Track Package and Cancel buttons.

9.2 Order Tracking

Step	Status	Icon
1	PENDING — Order Placed	bi-bag-check
2	CONFIRMED — Order Confirmed	bi-check-circle
3	SHIPPED — Shipped	bi-truck
4	DELIVERED — Delivered	bi-house-check

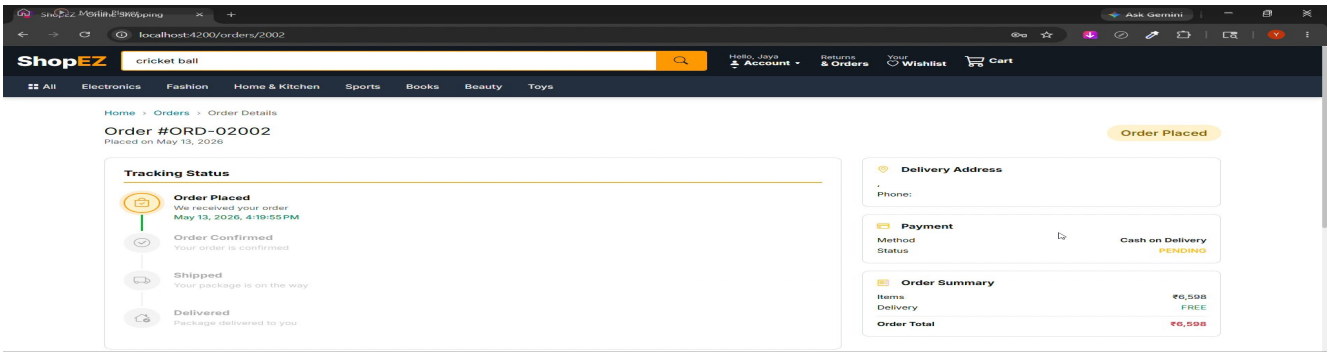


Figure 14: Order Tracking — Order #ORD-02002 (May 13, 2026). Tracking Status: Order Placed highlighted (green). Order Confirmed, Shipped, Delivered shown grey. Payment: Cash on Delivery, Status: PENDING. Total: ₹6,598.

10. Admin Panel

All /admin routes are protected by adminGuard which decodes the ADMIN role directly from the JWT payload — no API call needed. If the role is not ADMIN the user is redirected to home.

10.1 Admin Dashboard

The dashboard uses RxJS forkJoin to fire four HTTP requests in parallel and wait for all to complete before updating the stats cards. shareReplay(1) is used on the orders observable so the raw orders response is fetched only once and shared between the orders count and revenue sum calculations.

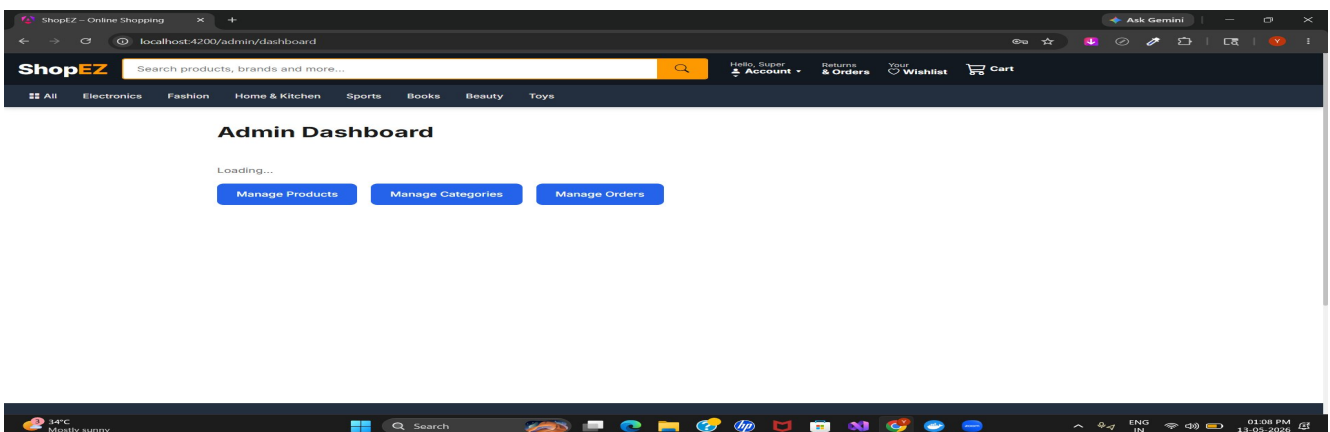


Figure 15: Admin Dashboard (localhost:4200/admin/dashboard) — forkJoin loading stats. Manage Products, Manage Categories, Manage Orders navigation buttons. Logged in as Super (ADMIN role).

10.2 Admin Manage Products

Clicking 'Add Product' or 'Edit' opens an inline form overlay with: Name, description, price, stock fields, Category select ([ngValue] binding for integer categoryId), Image file input (stores File object in selectedFile). On submit, createProduct() or updateProduct() sends FormData (not JSON) because the backend endpoint uses [FromForm] + IFormFile for multipart upload.

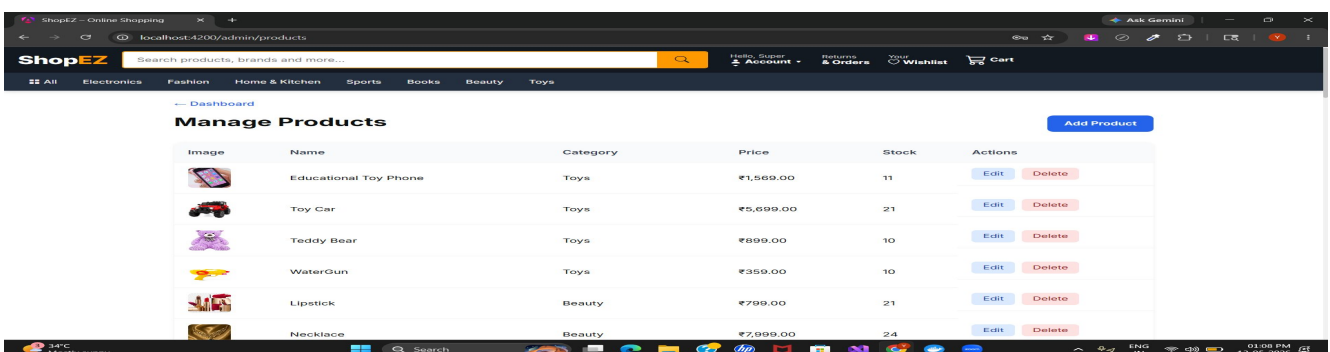


Figure 16: Admin Manage Products — table with product image thumbnails, Name, Category, Price (₹), Stock, Edit/Delete action buttons. 'Add Product' button top-right. Shows: Educational Toy Phone, Toy Car, Teddy Bear, WaterGun, Lipstick, Necklace.

10.3 Admin Manage Categories

CRUD table for product categories with inline Add/Edit form overlay. Category name uniqueness is enforced by the backend. Deleting a category that still has products is blocked by the backend (HTTP 400), which the frontend catches and displays.

10.4 Admin Manage Orders

Tabular view of all orders with a status dropdown per row. Selecting a new status immediately calls `adminService.updateOrderStatus(orderId, newStatus)` which sends `PATCH /api/orders/:id/status`. The table reloads after success to reflect the updated state.

11. Core Angular Concepts Used

Component & Service Tree



Core Services: `auth.service` · `cart.service` · `product.service` · `order.service` · `admin.service` · `auth-interceptor` · `guards`

Figure: Angular Component Tree — AppComponent root → 5 feature modules + Core Services bar

Route Guards

authGuard	adminGuard	guestGuard
auth-guard.ts	admin-guard.ts	guest-guard.ts
Checks <code>getToken()</code> No token → <code>/login?returnUrl=<current></code>	Decodes JWT payload directly (no API call), checks <code>role=ADMIN</code> Not ADMIN → redirect /	Checks <code>getToken()</code> Already logged in → redirect / (prevents re-visiting login)
Protects: Protected routes: <code>/cart</code> , <code>/checkout</code> , <code>/orders</code> , <code>/wishlist</code>	Protects: <code>/admin/**</code> all routes	Protects: <code>/login</code> , <code>/register</code>

Figure: Route Guards — authGuard, adminGuard, guestGuard — protecting different route groups

Guard	File	Logic
authGuard	auth-guard.ts	Checks <code>getToken()</code> — if no token, redirects to <code>/login?returnUrl=</code>
adminGuard	admin-guard.ts	Decodes JWT payload directly (no API call), checks <code>ClaimTypes.Role = ADMIN</code>
guestGuard	guest-guard.ts	Checks <code>getToken()</code> — if logged in, redirects to <code>/</code> (prevents visiting login when already authenticated)

11.1 HTTP Interceptor — AuthInterceptor

Registered globally in `app.config.ts` as `{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }`. It intercepts every outgoing HTTP request and clones it with an `Authorization: Bearer` header automatically. It also handles 401 errors intelligently — token expiry is checked by decoding the JWT `exp` claim at the time of the error.

11.2 RxJS BehaviorSubject — Reactive State

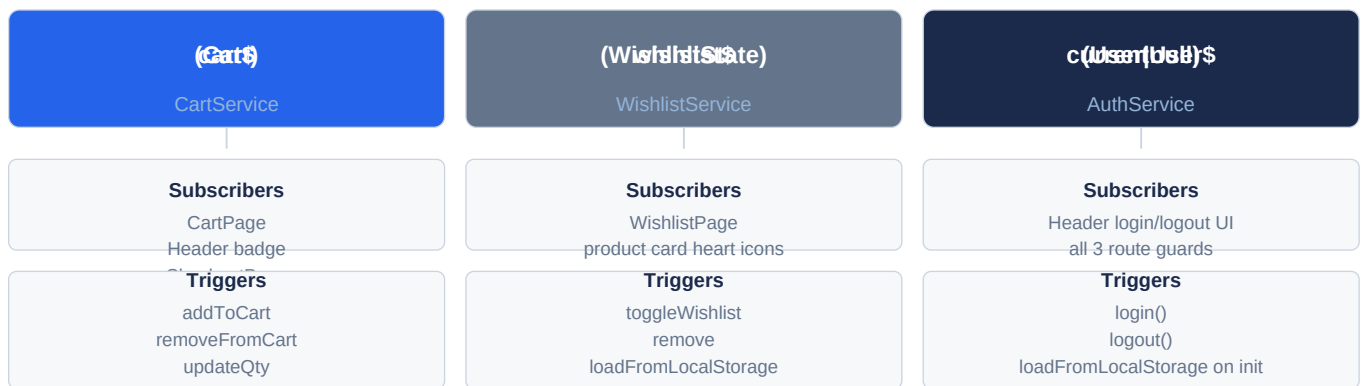


Figure: BehaviorSubject pattern — `cart$`, `wishlist$`, `currentUser$` live state across all components

11.3 takeUntil — Memory Leak Prevention

Every component that subscribes to an Observable uses the `takeUntil(destroy$)` pattern. A `Subject destroy$` fires in `ngOnDestroy`, completing all subscriptions. Without this, subscriptions to services like `cart$` would keep running after the component is removed from the DOM, causing memory leaks.

11.4 Client-Side Filtering

Instead of making an API call per filter change, all products are fetched once and stored in `allProducts[]`. Filters are applied in-memory via `Array.filter()` and `Array.sort()`. This pattern gives instantaneous filter response and is excellent for datasets up to ~1000 items.

11.5 API Response Normalisation

The .NET backend returns PascalCase JSON (`ProductId`, `CategoryName`). The `ProductService.mapProduct()` method uses the nullish coalescing operator (`??`) to support both PascalCase and camelCase. Similarly, `getProducts()` and `getMyOrders()` normalise all known .NET response envelope shapes to a consistent flat array before mapping.

12. Shared Components

Component	Selector	Purpose
Header	app-header	Sticky navbar. Subscribes to currentUser\$ (reactive login display) and cart\$ (badge count). Search bar navigates to /products?keyword. Category quick-links bar below
Loading Spinner	app-loading-spinner	Accepts message and optional overlay=true inputs. Overlay mode covers full screen during order processing
Alert Messages	app-alert-messages	Accepts message (string), type (success/danger/info/warning), emits dismiss. Auto-dismiss via setTimeout in parent components (3–4 seconds)
Rating	app-rating	Renders star icons (bi-star-fill, bi-star-half, bi-star) from a decimal rating value. Accepts size, count (review count), showCount inputs
Filter Sidebar	app-filter-sidebar	@Input() initialFilter, @Output() filterChange. Loads categories from API. Contains price preset chips, rating checkboxes, sort dropdown

13. Testing

13.1 Results — 58 Specs, 0 Failures

Test Suite	Specs	What Is Tested
CartService	24	addToCart, accumulate quantity, removeFromCart, updateQuantity, clearCart, totalItems, totalPrice, localStorage persistence, corrupted storage recovery, BehaviorSubject emission
Register Component	10	Phone pattern validation, email format, name min length, password strength scoring (0–4), strengthLabel output, isLoading flag, form validity
Login Component	9	Email validation, password length check (< 6 chars), form validity, error message state, isLoading flag, form submission prevention
adminGuard	3	No token → redirect /login; ADMIN role → allow access; CUSTOMER role → redirect /
App Component	2	App renders without errors, component creation
TOTAL	58	All Passed — 0 Failures

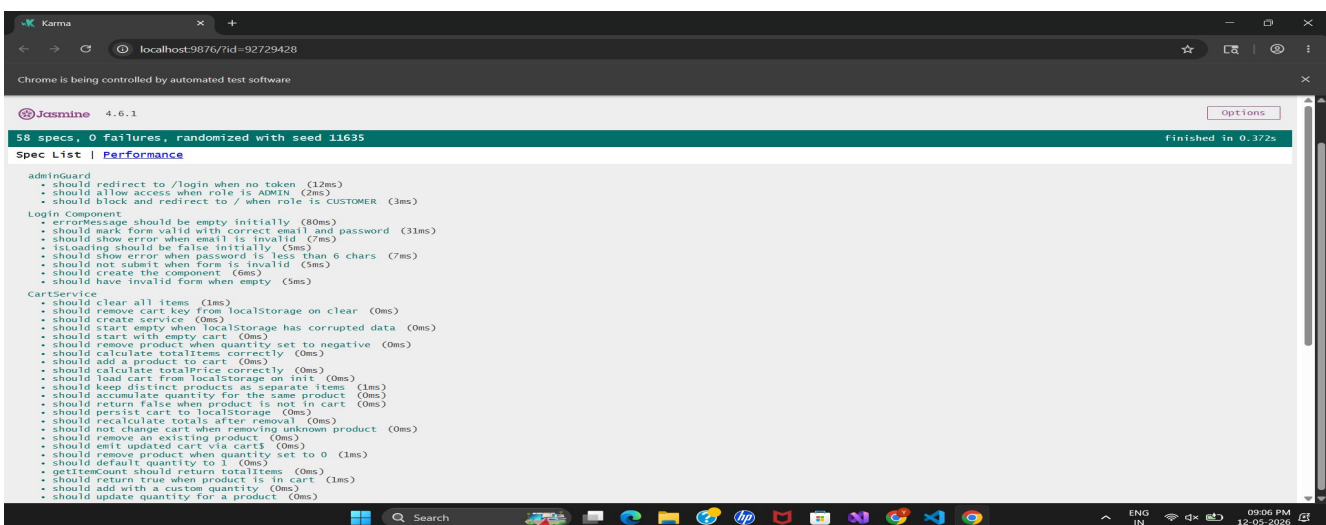


Figure 20: Full test run — adminGuard (3), Login Component (9), CartService (24), Register Component (10), App (2) — total 58 specs, 0 failures, finished in 0.372s

14. TypeScript Data Models

All interfaces are defined in `src/app/models/` and define the shape of data throughout the application:

Model File	Key Interfaces	Notable Fields
user.ts	User, AuthRequest, RegisterRequest, AuthResponse	role: 'CUSTOMER' 'ADMIN'
product.ts	Product, ProductFilter, PagedResponse	imageUrl (resolved to full URL by service), categoryId (optional)
category.ts	Category	id, name, description
cart.ts	Cart, CartItem	Cart.totalItems and Cart.totalPrice computed on every update
order.ts	Order, OrderItem, Address, OrderStatus, PaymentMethod	status: OrderStatus type union, paymentMethod: PaymentMethod type
wishlist.ts	WishlistItem, WishlistState	product (full object snapshot), addedAt timestamp
review.ts	Review, ReviewRequest	rating, title, comment, user: {id, name}

15. Environment & How to Run

15.1 Environment Configuration

Variable	Value
apiGatewayUrl	http://localhost:7000 — YARP API Gateway entry point
apiUrl	http://localhost:7000/api — All API requests routed through gateway
apiBaseUrl	http://localhost:7002 — Product image base URL (served directly from ProductService)
production	false — Development mode

15.2 Setup & Run Instructions

Step	Command / Action
1. Install Dependencies	npm install
2. Ensure Backend Running	Ports: 7000 (Gateway), 7001 (Auth), 7002 (Products), 7003 (Cart), 7004 (Orders)
3. Start Dev Server	ng serve -o
4. Open in Browser	http://localhost:4200
5. Admin Login	Email: admin@shopez.com Password: Admin@123
6. Run All Tests	ng test

16. Docker — Frontend Container

The Angular frontend is containerized alongside all backend services using Docker Compose. The Angular container runs the production build served via Nginx (or ng serve in development mode).

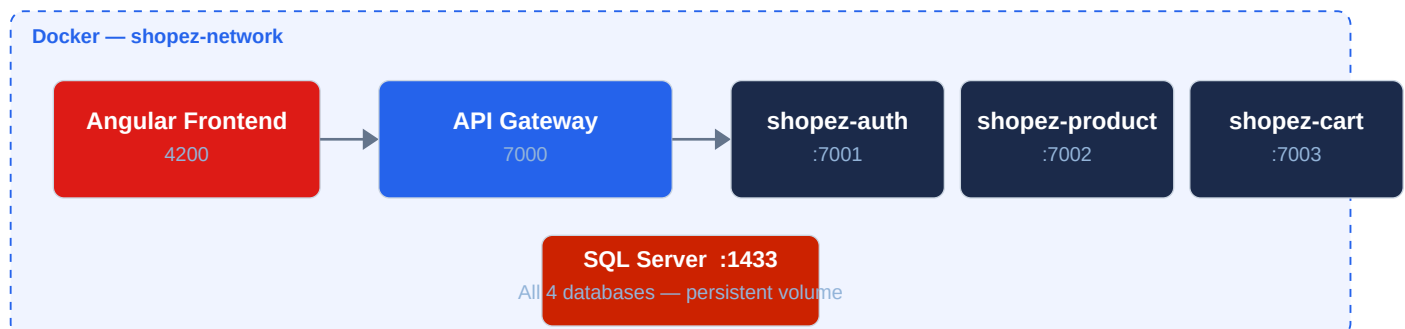


Figure: Docker — Angular Frontend container on shopez-network, connecting to API Gateway :7000

16.1 Angular Dockerfile

```

# Stage 1: Build Angular app
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build --prod

# Stage 2: Serve with Nginx
FROM nginx:alpine
COPY --from=build /app/dist/shopez /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
  
```

16.2 docker-compose.yml — Angular Service

```

angular-frontend:
  build:
    context: ./ShopEZ_Angular
    dockerfile: Dockerfile
  container_name: shopez-angular
  ports:
    - "4200:80"
  environment:
    - API_GATEWAY_URL=http://shopez-gateway:80
  depends_on:
    - shopez-gateway
  networks:
    - shopez-network
  
```

16.3 Key Docker Configuration Points

- SQL Server healthcheck — all backend services use depends_on with condition: service_healthy so they wait until SQL Server is ready.
- Shared network — all containers communicate on shopez-network bridge network using container names as hostnames (e.g., http://auth-service:80).
- Persistent volumes — sqlserver_data and product_images are named Docker volumes. Data survives container restarts. Only docker-compose down -v deletes them.
- Environment variables — JWT secrets, connection strings, and CORS origins injected as environment variables. No secrets hardcoded in source code.
- YARP Gateway routing — API Gateway receives upstream service addresses via environment variables.

16.4 Build & Run Commands

Command	Purpose
docker-compose up --build	Build images and start all containers
docker-compose up	Start all containers (no rebuild)
docker-compose down	Stop all containers (data preserved)
docker-compose down -v	Stop containers and delete volumes (data lost)
docker ps	List all running containers
docker logs shopez-angular	View logs for Angular container

17. Data Flow Diagram

User Login

Browser → LoginComponent → AuthService → Gateway /api/auth/login → AuthService :7001
POSTs credentials, receives JWT. Token stored in localStorage. BehaviorSubject emits user — header and guards update ins

Browse Products

ProductList → ProductService.getProducts() → Gateway /api/products → ProductService :7002
All products fetched once. Client-side filter pipeline (category, price, rating, keyword) on allProducts[] — no re-fetch

Add to Cart

ProductDetail → CartService.addToCart() → BehaviorSubject emits → localStorage updated
Immediate local state update. Cart badge in header refreshes instantly. No API call — cart is 100% client-side.

Place Order

Checkout → OrderService.placeOrder() → AuthInterceptor adds JWT → Gateway /api/orders → OrderService :7004
Server calculates TotalAmount server-side. On success: cart cleared, redirect to /orders?placed=true.

Admin Actions

AdminPanel → AdminService → Gateway (ADMIN JWT) → ProductService :7002 / OrderService :7004
adminGuard decodes JWT locally. Admin can CRUD products (multipart/form-data), update order status (PATCH), view all use

Figure: ShopEZ Frontend Data Flow — User → Angular Components → Services/State → AuthInterceptor → API Gateway → Microservices

17.1 Key Data Flow Paths

Flow	Path	Description
User Login	Browser → LoginComponent → AuthService → Gateway /api/auth/login → AuthService :7001	POSTs credentials, receives JWT. Token stored in localStorage. BehaviorSubject emits logged-in user — header and guards update instantly.
Browse Products	ProductList → ProductService.getProducts() → Gateway /api/products → ProductService :7002	All products fetched once. Client-side filter pipeline runs on cached allProducts[] — no re-fetch per filter change.
Add to Cart	ProductDetail → CartService.addToCart() → BehaviorSubject emits → localStorage updated	Immediate local state update. Cart badge in header refreshes instantly. No API call — cart is 100% client-side.
Place Order	Checkout → OrderService.placeOrder() → AuthInterceptor adds JWT → Gateway /api/orders → OrderService :7004	Server calculates TotalAmount server-side. On success: cart cleared, redirect to /orders?placed=true.

Flow	Path	Description
Admin Actions	AdminPanel → AdminService → Gateway (ADMIN JWT) → ProductService :7002 / OrderService :7004	adminGuard decodes JWT locally. Admin can CRUD products (multipart/form-data), update order status (PATCH).

18. Conclusion

ShopEZ's Angular frontend is a complete, production-quality Single Page Application that successfully demonstrates the full range of skills developed during the 3-month UpGrad CTS training program.

18.1 What Was Achieved

- Built 15+ Angular standalone components covering every user-facing feature of a real e-commerce platform
- Implemented the BehaviorSubject reactive state pattern for cart, wishlist, and authentication — enabling instant cross-component updates without prop-drilling
- Designed a client-side filtering architecture that gives instantaneous filter response without additional API calls
- Applied real-world Angular best practices: takeUntil memory leak prevention, route guards, HTTP interceptors, and reactive forms with validators
- Wrote 58 unit tests with Karma + Jasmine achieving 0 failures — covering CartService (24), Register (10), Login (9), adminGuard (3), App (2)
- Built a full Admin Panel with product image upload, category management, order status management, and user listing
- Normalised API responses defensively to handle both PascalCase (.NET default) and camelCase JSON formats

18.2 Challenges Overcome

Challenge	Solution Applied
Cross-component cart synchronisation	BehaviorSubject in CartService — all components subscribe to cart\$ and update simultaneously
.NET PascalCase JSON vs Angular camelCase	mapProduct() with ?? operator checks both ProductId and productId — defensive normalisation
Angular change detection missing async updates	ChangeDetectorRef.detectChanges() called explicitly after HTTP response in ProductDetails
JWT role case sensitivity	Role is uppercased on localStorage write — 'admin', 'Admin', 'ADMIN' all normalised to 'ADMIN'
Memory leaks from Observables	takeUntil(destroy\$) pattern applied in every component that subscribes to any Observable
Admin image upload (multipart)	FormData used instead of JSON — backend expects [FromForm] + IFormFile, not application/json

19. Future Scope

Enhancement	Description	Benefit
Real-Time Cart Sync to Backend	Move cart storage from localStorage to CartService API	Cross-device support, data consistency
Server-Side Pagination	Page buttons / infinite scroll replacing full-product-load	Scalability for large catalogues
Toast Notification System	Slide-in toast (top-right) replacing alert banners	Polished, non-blocking UX
Dark Mode	CSS variable-based theme switch persisted in localStorage	Accessibility, modern UX
WebSocket Order Updates	SignalR for live order status push to customers	Real-time transparency
Micro-Frontend Architecture	Split Admin Panel into separately deployable Angular app using Webpack 5 Module Federation	Independent deployment
Performance Budget CI	Lighthouse CI in GitHub Actions — fail build on regression	Quality assurance